

Accelerated Blahut–Arimoto Capacity Computation for Discrete Memoryless Channels

Amir Rasmi Al-Rashayda
 Department of Computer Engineering
 Birzeit University
 Birzeit, Palestine
 amir.alrashyda@gmail.com

Abstract—Channel capacity is a central quantity in information theory and is defined as the maximum mutual information between channel input and output. For general discrete memoryless channels (DMCs), capacity lacks a closed-form solution and is commonly computed numerically using the Blahut–Arimoto (BA) algorithm. This paper presents a rigorous and reproducible BA implementation in Python with a bound-gap termination criterion, validated against analytic capacities for the binary symmetric and binary erasure channels. We then investigate two acceleration strategies: (i) a monotone relaxed-update method with backtracking that enforces non-decreasing BA lower bounds, and (ii) a fast geometric-extrapolation method that preserves positivity while maintaining a single bound evaluation per iteration. Experiments on random DMCs show that the monotone method reduces iterations by approximately $1.58\times$ – $1.59\times$ but increases wall-clock time due to per-iteration overhead, whereas the fast method reduces iterations by about $1.30\times$ and improves runtime by approximately $1.17\times$ – $1.28\times$ at a tolerance of $I_U - I_L \leq 10^{-10}$ bits/use.

Index Terms—Information Theory, Channel Capacity, Discrete Memoryless Channel, Blahut–Arimoto Algorithm, Accelerated Convergence, Python Reproducibility.

I. INTRODUCTION

Channel capacity characterizes the maximum reliable communication rate supported by a channel. For a discrete memoryless channel (DMC) with transition probabilities $W(y|x)$, capacity is defined as

$$C = \max_{p(x)} I(X; Y), \quad (1)$$

where $p(x)$ is the input distribution and $I(X; Y)$ is the mutual information. While closed-form expressions exist for special channels (e.g., BSC and BEC), general DMCs require numerical computation. The classical solution is the Blahut–Arimoto (BA) algorithm.

A. Contributions

This work makes three contributions:

- 1) A tested Python implementation of BA with a rigorous termination rule using upper/lower capacity bounds.
- 2) Two accelerated BA variants: a monotone relaxed method and a fast geometric-extrapolation method.

This project was submitted as a course bonus deliverable. All experiments are reproducible in Python with unit tests.

Algorithm 1 Blahut–Arimoto with Bound-Gap Stopping

Require: Channel matrix $W(y|x)$, tolerance ε

- 1: Initialize $p(x) \leftarrow$ uniform
 - 2: **repeat**
 - 3: $q(y) \leftarrow \sum_x p(x)W(y|x)$
 - 4: $D(x) \leftarrow \sum_y W(y|x) \log \frac{W(y|x)}{q(y)}$
 - 5: $p(x) \leftarrow \frac{p(x)e^{D(x)}}{\sum_{x'} p(x')e^{D(x')}}$
 - 6: Compute bounds I_L, I_U
 - 7: **until** $I_U - I_L \leq \varepsilon$
 - 8: **return** $C \leftarrow \frac{1}{2}(I_L + I_U)$ and $p^*(x) \leftarrow p(x)$
-

- 3) A reproducible evaluation with tables and figures suitable for publication, including correctness validation and performance scaling.

II. BACKGROUND

A DMC is specified by $W(y|x)$ over finite alphabets $\mathcal{X}$ and $\mathcal{Y}$, with $\sum_y W(y|x) = 1$ for each x . Mutual information is

$$I(X; Y) = \sum_{x,y} p(x)W(y|x) \log_2 \frac{W(y|x)}{\sum_{x'} p(x')W(y|x')}. \quad (2)$$

Capacity is obtained by maximizing $I(X; Y)$ over all valid input distributions $p(x)$.

III. BLAHUT–ARIMOTO ALGORITHM

Given an input distribution $p(x)$, define the induced output distribution $q(y) = \sum_x p(x)W(y|x)$ and

$$D(x) = \sum_y W(y|x) \log \frac{W(y|x)}{q(y)}. \quad (3)$$

The BA update is

$$p_{\text{BA}}(x) \propto p(x) \exp(D(x)). \quad (4)$$

A. Bound-Gap Termination

BA yields computable bounds $I_L \leq C \leq I_U$. We stop when the gap satisfies

$$I_U - I_L \leq \varepsilon, \quad (5)$$

and report $C \approx \frac{1}{2}(I_L + I_U)$.

Algorithm 2 Fast Accelerated BA (Geometric Extrapolation)

Require: Current p , BA step p_{BA} , parameter $\alpha > 1$

- 1: $p_{\text{new}}(x) \leftarrow p(x)^{1-\alpha} p_{\text{BA}}(x)^\alpha$
- 2: **Normalize** $p_{\text{new}} \leftarrow p_{\text{new}} / \sum_x p_{\text{new}}(x)$
- 3: **return** p_{new}

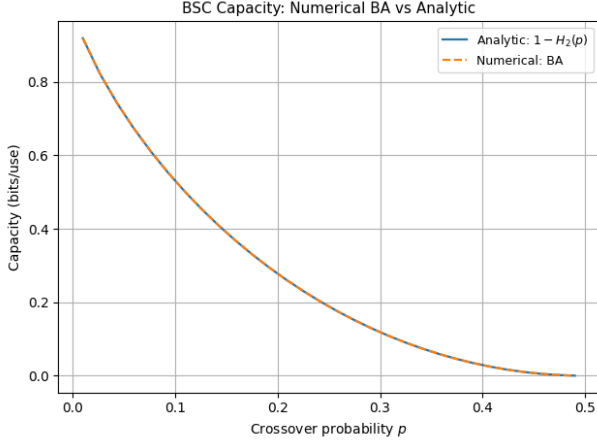


Fig. 1: Validation on BSC: numerical BA capacity matches analytic capacity $1 - H_2(p)$.

IV. ACCELERATION METHODS

We evaluate two acceleration strategies.

A. Monotone Relaxed Updates

We compute the BA step p_{BA} and apply a relaxed update. A backtracking rule ensures the BA lower bound does not decrease. This reduces iteration count but increases per-iteration cost.

B. Fast Geometric Extrapolation

To retain near-baseline per-iteration complexity, we use a geometric extrapolation update:

$$p_{\text{new}}(x) \propto p(x)^{1-\alpha} p_{\text{BA}}(x)^\alpha, \quad \alpha > 1. \quad (6)$$

This preserves positivity and requires only one bound evaluation per iteration.

V. EXPERIMENTAL SETUP

We validate correctness using closed-form capacities for BSC and BEC, then benchmark on random DMCs generated by row-normalized i.i.d. positive samples with a fixed seed. The termination tolerance is set to $I_U - I_L \leq 10^{-10}$ bits/use.

VI. RESULTS

A. Correctness Validation

We first validate numerical correctness on channels with known closed-form capacities. For the binary symmetric channel (BSC) with crossover probability p , capacity is $C_{\text{BSC}} = 1 - H_2(p)$, where $H_2(\cdot)$ is the binary entropy function. Fig. 1 shows that the capacity estimated by our BA implementation matches the analytic curve across the tested

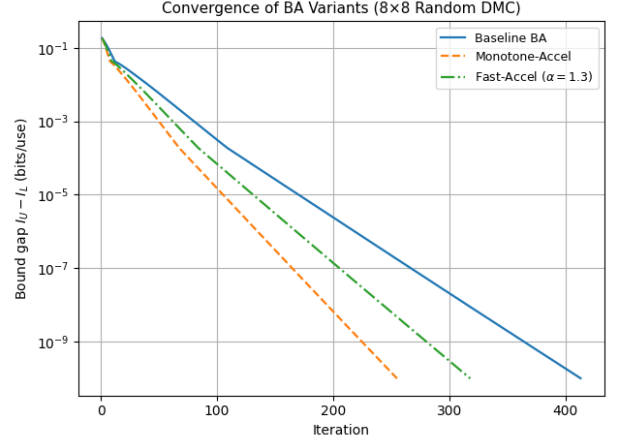


Fig. 2: Convergence of BA variants on a representative 8×8 random DMC using the bound gap $I_U - I_L$.

range of p . This agreement confirms that (i) the BA update, (ii) the bound computation, and (iii) the stopping criterion based on the bound gap are implemented correctly.

B. Convergence Behavior

The BA algorithm provides both a lower bound I_L and an upper bound I_U on capacity, allowing a rigorous termination rule. We therefore measure convergence using the *bound gap* $I_U - I_L$ rather than changes in $p(x)$ or the mutual information value alone. Fig. 2 plots the bound gap versus iteration for a representative 8×8 random DMC at tolerance $\varepsilon = 10^{-10}$ bits/use. After a short transient, the gap decreases approximately exponentially (nearly linear on the semilog axis), which is consistent with the classical behavior of BA-type fixed-point iterations. Both accelerated variants reduce the number of iterations required to reach the target tolerance, indicating faster contraction in the relevant region of the simplex.

C. Benchmark Summary and Interpretation

Table I reports averaged iteration counts and runtimes under identical termination tolerance $I_U - I_L \leq 10^{-10}$ bits/use. Across all tested sizes, the final capacity estimates agree up to numerical precision (maximum absolute differences on the order of 10^{-12} bits/use in our benchmarks), indicating that acceleration does not degrade accuracy at the chosen tolerance.

Two distinct effects explain the performance trends in Table I:

Iteration reduction: The monotone accelerated method reduces iteration count by approximately $1.58\times$ – $1.59\times$ across tested sizes. For example, at $(8, 8)$ it reduces iterations from 13379.9 to 8410.3. The fast accelerated method also reduces iterations by roughly $1.30\times$ (e.g., 13379.9 to 10291.2 at $(8, 8)$). These results confirm that both acceleration strategies improve convergence when measured in iterations.

Per-iteration overhead: Despite fewer iterations, the monotone method is slower in wall-clock time (e.g., 1.3425 s vs 1.0469 s at $(8, 8)$). This is expected because enforcing monotonicity typically requires additional work per iteration (e.g., extra checks and auxiliary evaluations). In contrast,

TABLE I: Benchmark Summary on Random DMCs ($\varepsilon = 10^{-10}$ bits/use).

(m, n)	Trials	It. BA	It. Mono	It. Fast	Sec. BA	Sec. Mono	Sec. I
(4,4)	30	1813.2	1142.4	1394.3	0.0992	0.1363	0.08
(8,8)	20	13379.9	8410.3	10291.2	1.0469	1.3425	0.81
(16,16)	10	8331.3	5266.1	6407.5	0.8687	1.1587	0.70

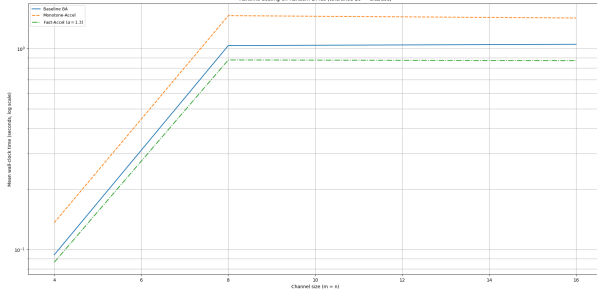


Fig. 3: Runtime scaling on random DMCs at tolerance $I_U - I_L \leq 10^{-10}$ bits/use.

the fast method improves wall-clock time (e.g., 0.8193 s vs 1.0469 s at (8, 8)) because it keeps per-iteration complexity close to baseline BA while still accelerating convergence. The practical takeaway is that monotonicity guarantees can reduce iteration count but may not be favorable when per-iteration cost dominates, whereas geometric extrapolation offers a better speed–accuracy trade-off in this implementation.

D. Scaling and Runtime

Fig. 3 visualizes mean runtime versus channel size and complements Table I. The fast method remains consistently faster than baseline BA across the tested sizes, while the monotone method remains slower due to its higher per-iteration cost. Although absolute runtimes depend on hardware and implementation details, the relative ranking under an identical tolerance and benchmarking protocol is stable, indicating that the fast accelerated method is the most practical improvement for time-constrained workflows.

E. Optimal Input Distribution on Asymmetric DMCs

Capacity is achieved by an input distribution $p^*(x)$ that maximizes mutual information. For symmetric channels this distribution is often uniform, but for asymmetric channels it is generally non-uniform. Fig. 4 visualizes $p^*(x)$ for an asymmetric 3×3 DMC. The resulting $p^*(x)$ is clearly non-uniform, illustrating a core information-theoretic concept that motivates numerical capacity computation beyond special symmetric cases. Importantly, baseline BA and the fast accelerated method converge to the same $p^*(x)$ (up to numerical precision), reinforcing that acceleration changes the convergence path but not the optimum.

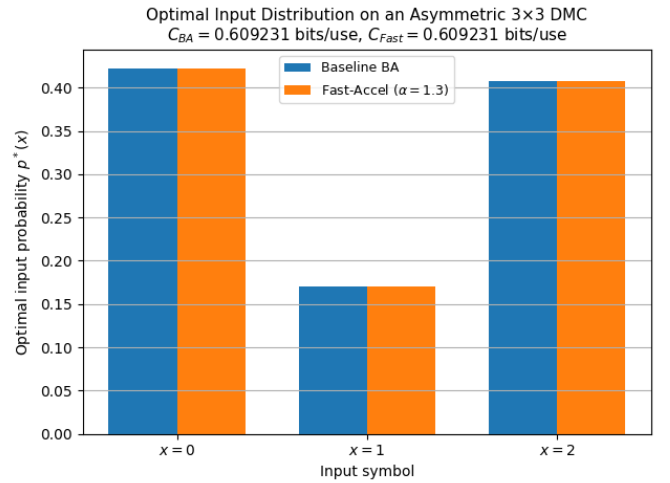


Fig. 4: Capacity-achieving input distribution $p^*(x)$ for an asymmetric 3×3 DMC is generally non-uniform.

VII. CONCLUSION

We presented a rigorous BA implementation with bound-gap termination and validated it against analytic results for standard channels. We then evaluated two acceleration strategies. The monotone variant consistently reduced iterations but increased runtime due to additional per-iteration checks. The fast geometric-extrapolation variant improved both iteration count and wall-clock time while preserving numerical accuracy. Future work includes systematic tuning of α , larger channel families, and acceleration methods with provable complexity benefits.

REFERENCES

- [1] R. E. Blahut, “Computation of channel capacity and rate-distortion functions,” *IEEE Trans. Inf. Theory*, vol. 18, no. 4, pp. 460–473, Jul. 1972.
- [2] S. Arimoto, “An algorithm for computing the capacity of arbitrary discrete memoryless channels,” *IEEE Trans. Inf. Theory*, vol. 18, no. 1, pp. 14–20, Jan. 1972.
- [3] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [4] Y. Yu, “Squeezing the Arimoto–Blahut algorithm for faster convergence,” *IEEE Trans. Inf. Theory*, vol. 56, no. 7, pp. 3149–3157, Jul. 2010.
- [5] G. Matz and P. Duhamel, “Information geometric formulation and interpretation of accelerated Blahut–Arimoto-type algorithms,” in *Proc. IEEE ITW*, 2004, pp. 66–70.